Laboratorio 5

Andrea Pérez
Santiago Ortiz
Dario Sanchez

Este laboratorio consistió en implementar una red neuronal convolucional para clasificar imágenes de frutas y verduras, utilizando el dataset Fruits-360 y el framework TensorFlow/Keras. El objetivo principal fue comparar el desempeño de dos arquitectuas distintas: una red diseñada desde cero y una red preentrenada usando transfer learning, para determinar cuál ofrece mejores resultados en términos de precisión y eficiencia.

Los pasos principales del trabajo fueron: cargar y preprocesar el dataset, entrenar ambas redes, evaluarlas con métricas de clasificación y matrices de confusión, y finalmente analizar los resultados para identificar cuál modelo es más adecuado para este problema.

Dataset

El dataset utilizado corresponde al dataset Fruits-360 disponible en Kaggle. Se trata de una base de datos amplia de alrededor de 90 clases de frutas y verduras, con imágenes de 100 x 100 pixeles en formato RGB. Para este laboratorio se trabajó con un subconjunto de 10 clases, específicamente las siguientes:

- Apricot
- Avocado
- Banana
- Cherry 1
- Cocos
- Kiwi
- Orange
- Pineapple
- Tomato 1
- Watermelon

Las imágenes están organizadas en carpetas por clase, con un total de 22688 imágenes para entrenamiento y 7574 para validación. Las clases están bastante balanceadas, con cantidades de entre 2000 y 2500 imágenes por clase, lo cual es un punto a favor para el entrenamiento.

Preprocesamiento

Se aplicaron las siguientes transformaciones a las imágenes antes de ingresarlas a la red:

- Reescalamiento: dividir los valores de pixeles por 255 para llevarlos al rango [0, 1].
- Data augmentation: rotaciones hasta 20°, desplazamientos horizontales y verticales del 10%, zoom del 10% y volteo horizontal, aplicados solo sobre el conjunto de entrenamiento.

Arquitectura 1: CNN desde cero

Se diseñó una red convolucional con 3 bloques convolucionales, cada uno con una capa Conv2D seguida de BatchNormalization, ReLU y MaxPooling2D, y luego una capa Dense de 128 neuronas con Dropout 0.5 antes de la capa de salida con 10 neuronas y activación softmax.

Los hiperparámetros fueron: optimizador Adam con learning rate 0.001, batch size 32, y hasta 30 épocas con EarlyStopping (paciencia 5) y ReduceLROnPlate (factor 0.5, paciencia 3).

Arquitectura 2: Transfer learning con MobileNetV2

Se usó MobileNetV2 como base convolucional, con los pesos de ImageNet congelados. Sobre la salida de MobileNetV2 se agregaron una capa GlobalAveragePooling2D, una Dense de 128 neuronas con ReLU, Dropout 0.5 y la capa de salida con 10 neuronas softmax.

Los hiperparámetros fueron: optimizador Adam con learning rate 0.001, batch size 32, y hasta 30 épocas con EarlyStopping (paciencia 5) y ReduceLROnPlate (factor 0.5, paciencia 3).

Resultados

Modelo	Accuracy	Precision	Recall	F1
CNN desde cero	1.0000	1.0000	1.0000	1.0000
Transfer learning (MobileNetV2)	1.0000	1.0000	1.0000	1.0000

Ambos modelos alcanzaron valores perfectos en todas las métricas sobre el conjunto de validación. Esto es inusual y sugiere que el problema es muy sencillo para ambas arquitecturas, o que las imágenes de validación son muy similares a las de entrenamiento.

Conclusiones

- Ambas arquitectuas funcionan muy bien para este problema, con métricas perfectas en el conjunto de validación.
- Transfer learning converge más rápido y requiere menos épocas para estabilizarse, lo cual es una ventaja en tiempo de entrenamiento.
- Dado que ambos modelos alcanzan el mismo desempeño, la elección depende de otros factores: la CNN desde cero es más pequeña y rápida de entrenar, mientras que MobileNetV2 ofrece mejor generalización a datos más complejos.
- Para este dataset, la CNN desde cero es suficiente y es la opción más eficiente.
- Se recomienda probar con datasets más difíciles (más clases, fondos complejos, iluminación variada) para evaluar si la ventaja de transfer learning se hace más evidente.

Bibliografía

Kaggle. Fruits-360 dataset. https://www.kaggle.com/datasets/moshesh/fruits-360
Keras Applications. MobileNetV2. https://keras.io/keras_applications/#mobilenet
TensorFlow Hub. https://tfhub.dev/

Anexos

```
!pip install -q kaggle
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json
!kaggle datasets download -d moshesh/fruits-360
!unzip -q fruits-360.zip -d /content/fruits360
!ls /content/fruits360/Fruits-360-Small/Training | head -20
```

```
pip install -q kaggle
mkdir: cannot create directory '/root/.kaggle': File exists
cp: cannot stat '/content/kaggle.json': No space left on device
chmod: cannot access '/root/.kaggle/kaggle.json': No space left on device
Dataset URL: https://kaggle.com/datasets/moshesh/fruits-360
unzip:  cannot find or open fruits-360.zip, fruits-360.zip.zip or fruits-360.zip.zipzip.
ls: cannot access '/content/fruits360/Fruits-360-Small/Training': No space left on device
```

```
import os, shutil, random, glob
from google.colab import drive
drive.mount('/content/drive')

!kaggle datasets download -d moshesh/fruits-360
!unzip -q fruits-360.zip -d /content/fruits360

# Copiar a Drive
!cp -r /content/fruits360 /content/drive/MyDrive/fruits360

# Ver estructura
!ls /content/drive/MyDrive/fruits360/Fruits-360-Small/Training | head -20
```

```
Mounted at /content/drive
Dataset URL: https://kaggle.com/datasets/moshesh/fruits-360
unzip:  cannot find or open fruits-360.zip, fruits-360.zip.zip or fruits-360.zip.zipzip.
cp: cannot stat '/content/fruits360': No space left on device
ls: cannot access '/content/drive/MyDrive/fruits360/Fruits-360-Small/Training': Transport endpoint is not connected
```